50.007 Machine Learning

Hidden Markov Model

(Continue…)

Roy Ka-Wei Lee
Assistant Professor, ISTD, SUTD

SINGAPORE UNIVERSITY OF TECHNOLOGY AND DESIGN

Learning Objectives

You should be able to know:

- What is decoding for the HMM?
- Why do we need the Viterbi algorithm for decoding?
- How to perform decoding using the Viterbi algorithm?
- What is the space and time complexity of the Viterbi algorithm?

HMM - Supervised Learning (Recap)

We estimate the parameters of the HMM by performing Maximum Likelihood Estimation.

$$p(\mathbf{x}, \mathbf{y}) = \underbrace{\prod_{j=0}^{n} a_{y_j, y_{j+1}}}_{\text{Transition probabilities}} \times \underbrace{\prod_{j=1}^{n} b_{y_j}(x_j)}_{\text{Emission probabilities}}$$

By setting the derivative of log-likelihood *w.r.t.* each individual parameter to zero, we can get a close-form estimate for each parameter:

Emission matrix

transition matrix

$$a_{u,v} = \frac{\text{count}(u,v)}{\text{count}(u)}$$

Number of times we see a transition from u to v

Number of times we see the state u in the training set

$$b_u(o) = \frac{\text{count}(u \rightarrow o)}{\text{count}(u)}$$

Number of times we see observation o generated from u

Number of times we see the state u in the training set

HMM (Example)

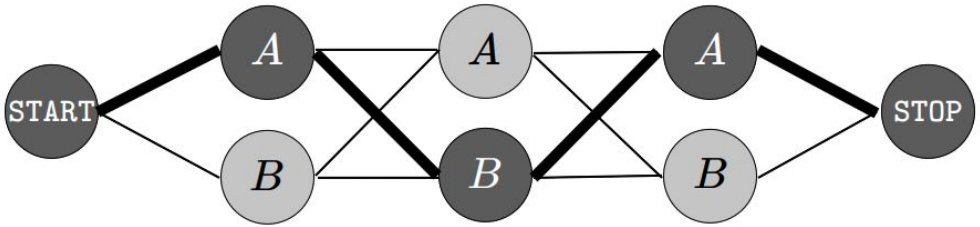
$a_{u,v}$

$u \backslash v$	A	B	STOP
START	1.0	0.0	0.0
A	0.5	0.5	0.0
B	0.0	0.8	0.2

$b_u(o)$

$u \backslash o$	*"the"*	*"dog"*
A	0.9	0.1
B	0.1	0.9

$$(\mathbf{x}, \mathbf{y}) = \text{the}/A, \text{dog}/B, \text{the}/A$$



$$a_{\text{START},A} \times b_A(\text{“the”}) \times a_{A,B} \times b_B(\text{“dog”}) \times a_{B,A} \times b_A(\text{“the”}) \times a_{A,\text{STOP}} = 0$$

HMM (Example)

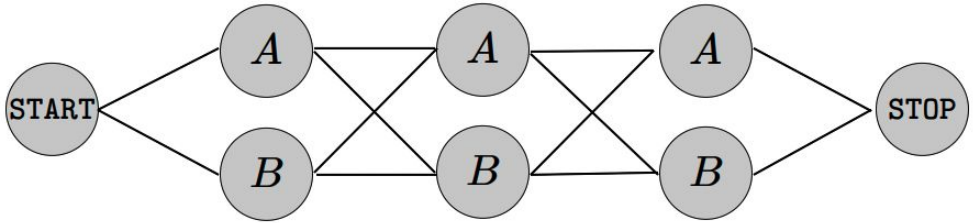
$a_{u,v}$

$u \backslash v$	A	B	STOP
START	1.0	0.0	0.0
A	0.5	0.5	0.0
B	0.0	0.8	0.2

$b_u(o)$

$u \backslash o$	*"the"*	*"dog"*
A	0.9	0.1
B	0.1	0.9

$$(\mathbf{x}, \mathbf{y}) = \text{the}/B, \text{dog}/B, \text{the}/B$$

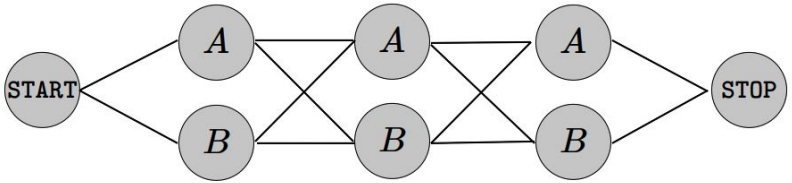


Which label sequence y is the most probable given the word sequence x?

Decoding

HMM - Decoding

$$(\mathbf{x}, \mathbf{y}) = \text{the}/B, \text{dog}/B, \text{the}/B$$



- Given the HMM (with estimated parameters) and an observation sequence **x**, find the most probable sequence of hidden states **y^*** for the given observation sequence **x**.

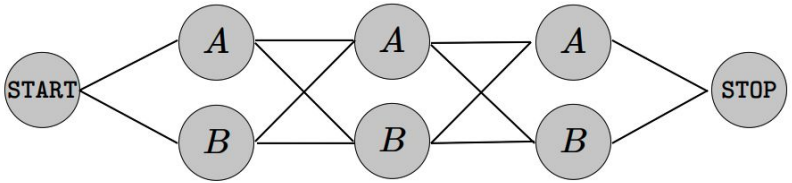
$$\mathbf{y}^* = \arg\max_{\mathbf{y}} p(\mathbf{y}|\mathbf{x})$$

↓

Return the parameters of y which gives the max. $P(y|x)$

HMM - Decoding

$$(\mathbf{x}, \mathbf{y}) = \text{the}/B, \text{dog}/B, \text{the}/B$$



- Given the HMM (with estimated parameters) and an observation sequence **x**, find the most probable sequence of hidden states **y^*** for the given observation sequence **x**.

$$\mathbf{y}^* = \arg\max_{\mathbf{y}} p(\mathbf{y}|\mathbf{x})$$

$$= \arg\max_{\mathbf{y}} p(\mathbf{x}, \mathbf{y})/p(\mathbf{x})$$

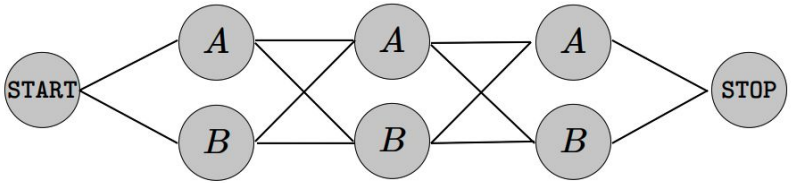
Bayes Theorem

will be constant ∵ it is the P of observing "the", "Dog", "the"

⇒ essentially able to drop

HMM - Decoding

$$(\mathbf{x}, \mathbf{y}) = \text{the}/B, \text{dog}/B, \text{the}/B$$

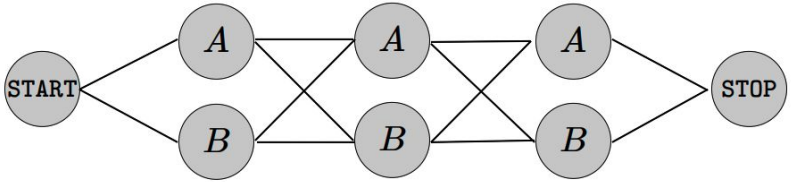


- Given the HMM (with estimated parameters) and an observation sequence **x**, find the most probable sequence of hidden states **y^*** for the given observation sequence **x**.

$$\begin{aligned}\mathbf{y}^* &= \arg\max_{\mathbf{y}} p(\mathbf{y}|\mathbf{x}) \\ &= \arg\max_{\mathbf{y}} p(\mathbf{x}, \mathbf{y})/p(\mathbf{x}) \\ &= \arg\max_{\mathbf{y}} p(\mathbf{x}, \mathbf{y})\end{aligned}$$

HMM - Decoding

$$(\mathbf{x}, \mathbf{y}) = \text{the}/B, \text{dog}/B, \text{the}/B$$



- Given the HMM (with estimated parameters) and an observation sequence **x**, find the most probable sequence of hidden states **y^*** for the given observation sequence **x**.

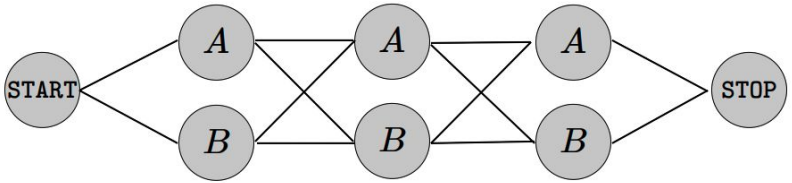
$$\begin{aligned}\mathbf{y}^* &= \arg\max_{\mathbf{y}} p(\mathbf{y}|\mathbf{x}) \\ &= \arg\max_{\mathbf{y}} p(\mathbf{x}, \mathbf{y})/p(\mathbf{x}) \\ &= \arg\max_{\mathbf{y}} p(\mathbf{x}, \mathbf{y})\end{aligned}$$

Brute forcing

We can try one **y** at a time, and see which gives the highest score!

HMM - Decoding

$$(\mathbf{x}, \mathbf{y}) = \text{the}/B, \text{dog}/B, \text{the}/B$$



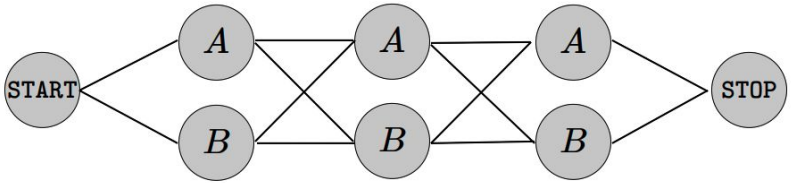
- Given the HMM (with estimated parameters) and an observation sequence **x**, find the most probable sequence of hidden states **y^*** for the given observation sequence **x**.

$$\begin{aligned}\mathbf{y}^* &= \arg\max_{\mathbf{y}} p(\mathbf{y}|\mathbf{x}) \\ &= \arg\max_{\mathbf{y}} p(\mathbf{x}, \mathbf{y})/p(\mathbf{x}) \\ &= \arg\max_{\mathbf{y}} p(\mathbf{x}, \mathbf{y})\end{aligned}$$

We can try one **y** at a time, and see which gives the highest score!

HMM - Decoding

$$(\mathbf{x}, \mathbf{y}) = \text{the}/B, \text{dog}/B, \text{the}/B$$



$$\mathbf{y}^* = \arg\max_{\mathbf{y}} p(\mathbf{x}, \mathbf{y})$$

Number of words in the sentence → # of words

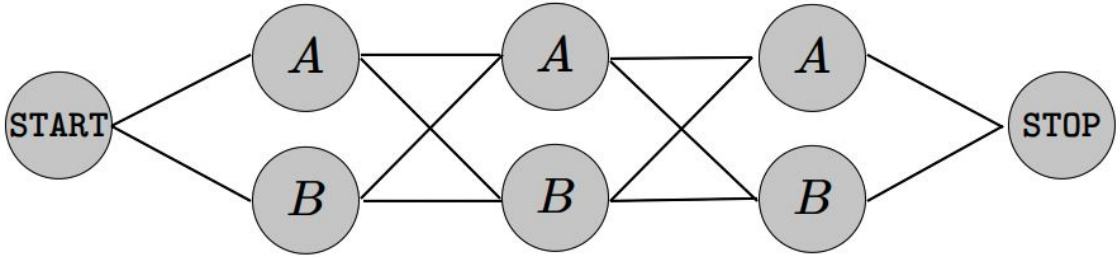
There are $O(|\mathcal{T}|^n)$ possible $\mathbf{y}$'s!

Number of possible tags at each word/position

2^3 # of hidden state

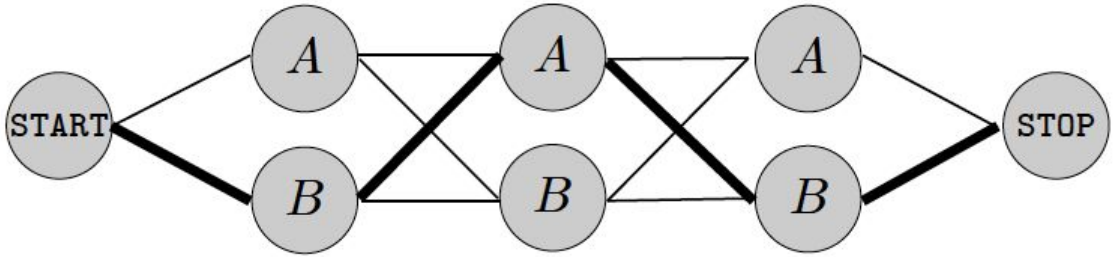
Finding Highest Scoring Path

- We aim to develop a more efficient approach for finding the most probable label sequence, or, the the highest scoring path connecting START and STOP.



Finding Highest Scoring Path

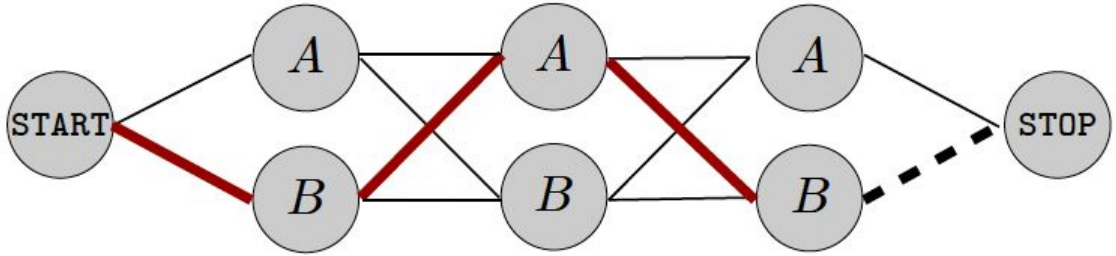
- We aim to develop a more efficient approach for finding the most probable label sequence, or, the the highest scoring path connecting START and STOP.



Let's assume this is the highest scoring path.

Finding Highest Scoring Path

- We aim to develop a more efficient approach for finding the most probable label sequence, or, the the highest scoring path connecting START and STOP.



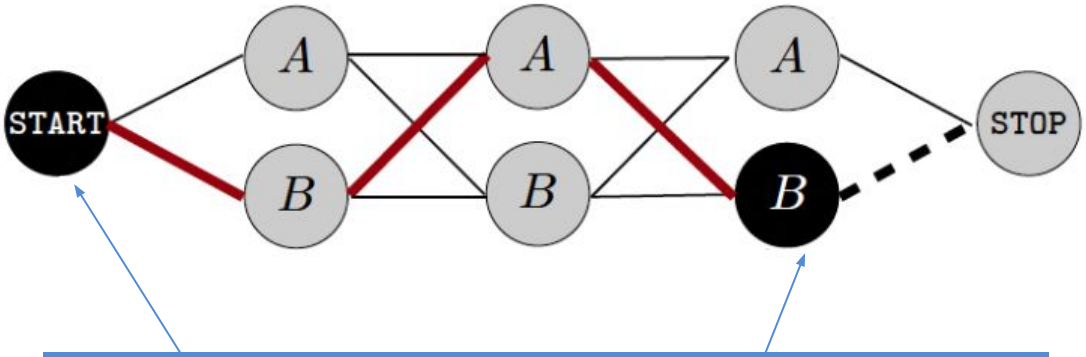
Let's assume this is the highest scoring path.

Then, what can we say about this partial path?

(What types of properties do we know for this path?)

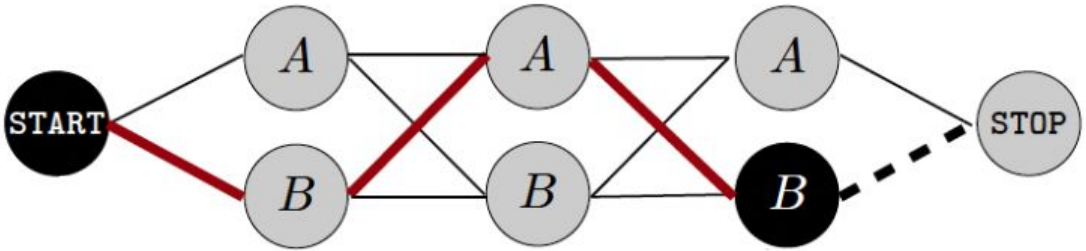
Finding Highest Scoring Path

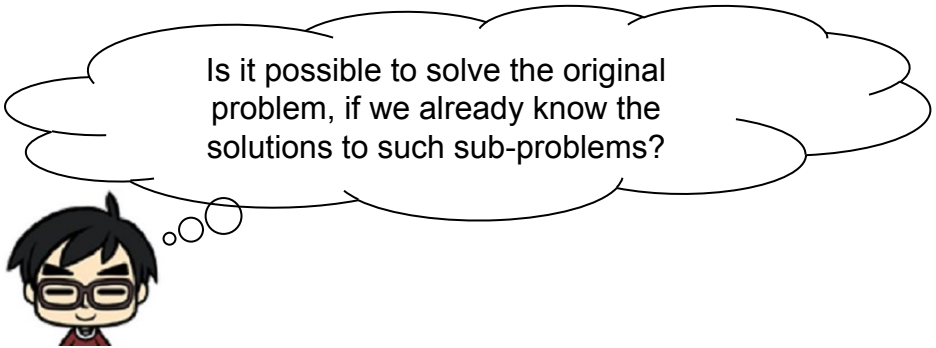
- We aim to develop a more efficient approach for finding the most probable label sequence, or, the the highest scoring path connecting START and STOP.



Finding Highest Scoring Path

- We aim to develop a more efficient approach for finding the most probable label sequence, or, the the highest scoring path connecting START and STOP.

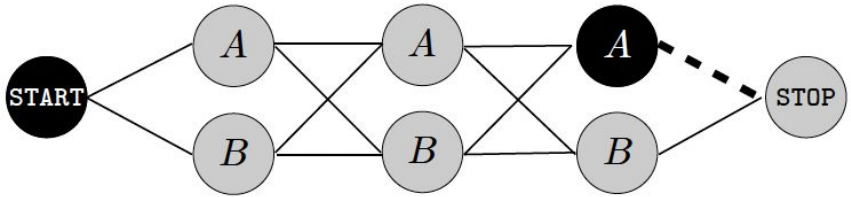




Finding Highest Scoring Path

Case A

The second last node in the highest scoring path is A.



Find the highest scoring path from START to A at position n

A single edge from node A to STOP

Finding Highest Scoring Path

Case A

The second last node in the highest scoring path is A.

Find the highest scoring path from START to A at position n

A single edge from node A to STOP

Finding Highest Scoring Path

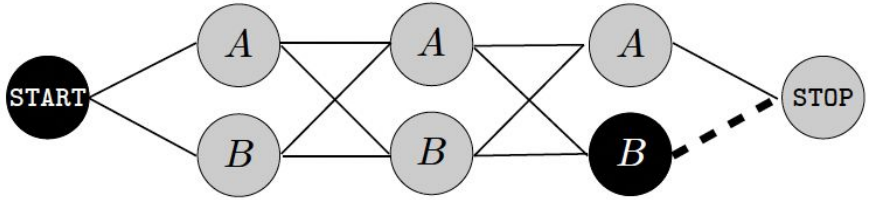
Case A
The second last node in the highest scoring path is A.

Find the highest scoring path from START to A at position n

+

A single edge from node A to STOP

Case B
The second last node in the highest scoring path is B.



Find the highest scoring path from START to B at position n

+

A single edge from node B to STOP

Finding Highest Scoring Path

Case A
The second last node in the highest scoring path is A.

Find the highest scoring path from START to A at position n

A single edge from node A to STOP

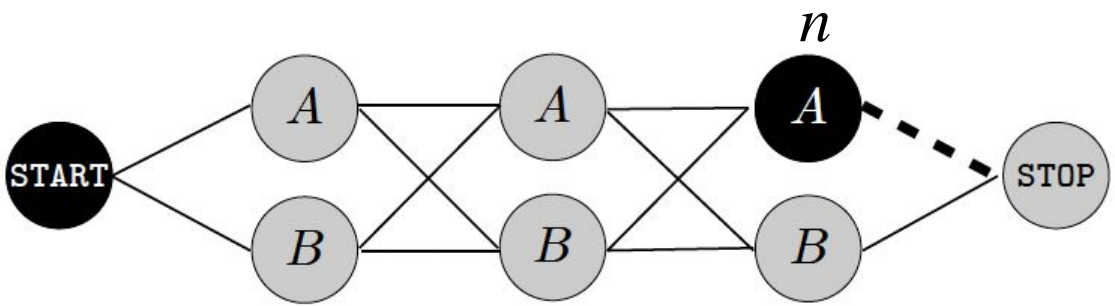
Case B
The second last node in the highest scoring path is B.

Find the highest scoring path from START to B at position n

A single edge from node B to STOP

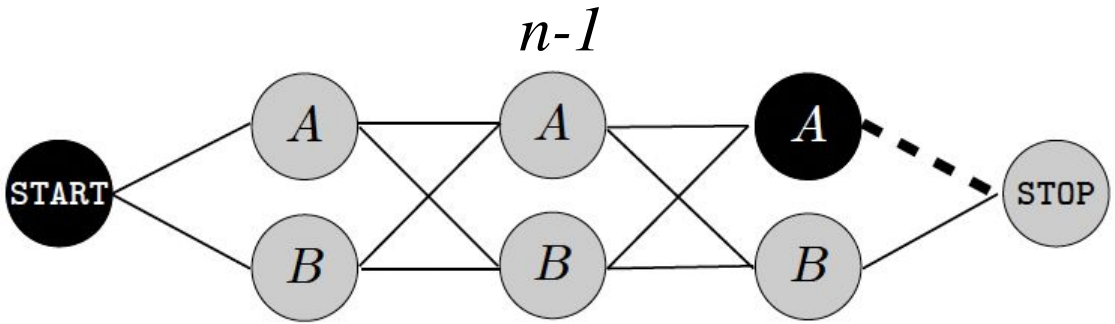
The highest scoring path from START to STOP: PATH 1 or PATH 2?

Finding Highest Scoring Path



How do you find the highest scoring path from START to node A at position n?

Finding Highest Scoring Path



How do you find the highest scoring path from START to node A at position n?

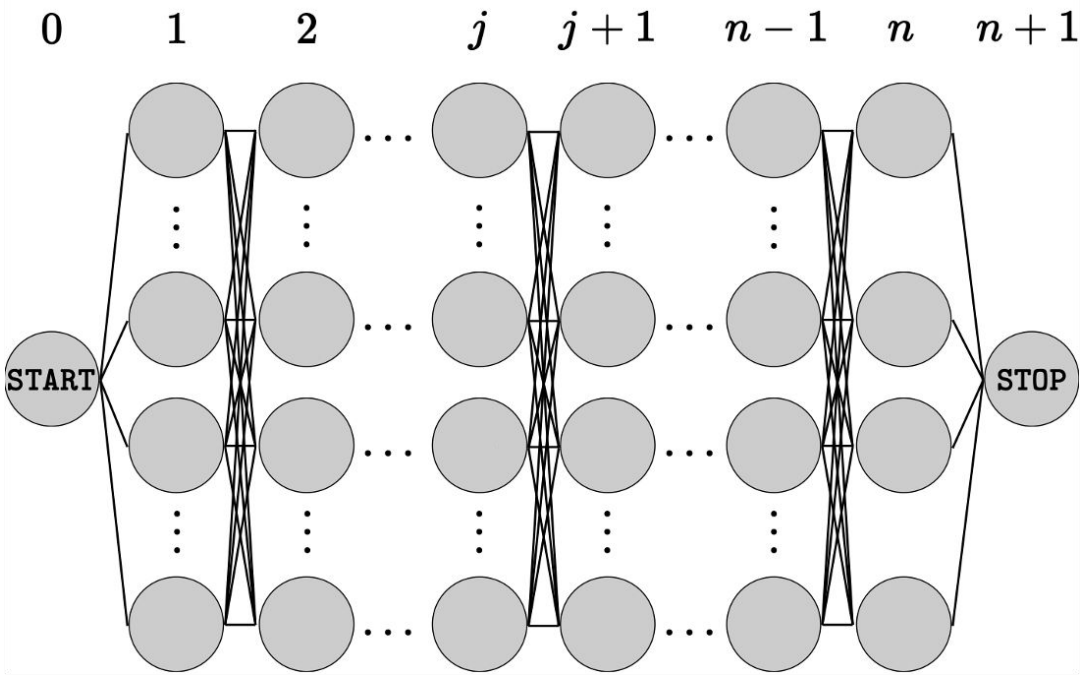
We shall again rely on the partial paths from START to the two nodes at position $(n-1)$

Dynamic Programming

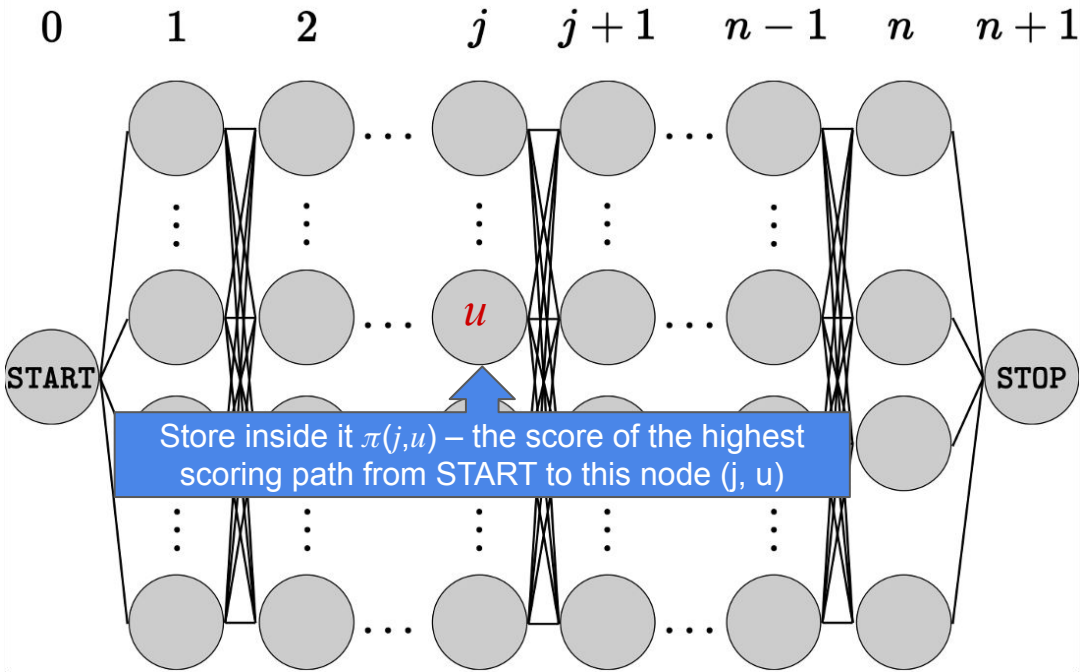
Question


Now, can we develop an efficient algorithm based on such observations?

Viterbi Algorithm



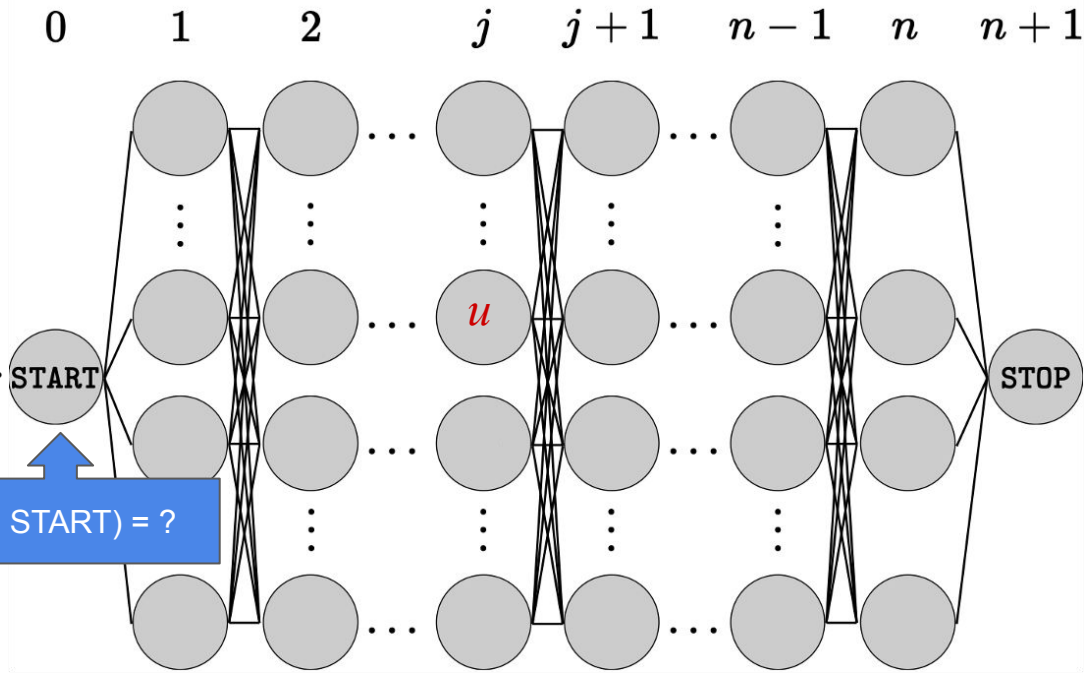
Viterbi Algorithm



$$\pi(j,u) = \max_{y_1,\ldots,y_{j-1}} \{p(x_1,\ldots,x_j, y_0 = \text{START}, y_1,\ldots,y_{j-1}, y_j = u)\}$$

storing the max. value instead of argmax.

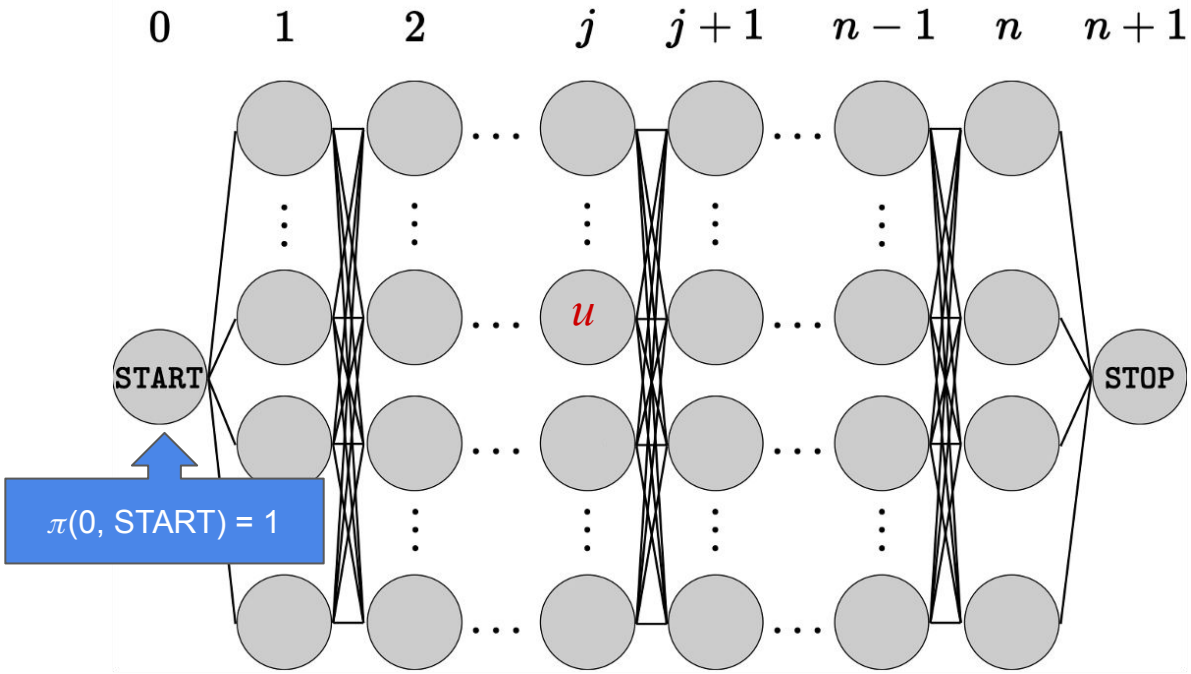
Viterbi Algorithm



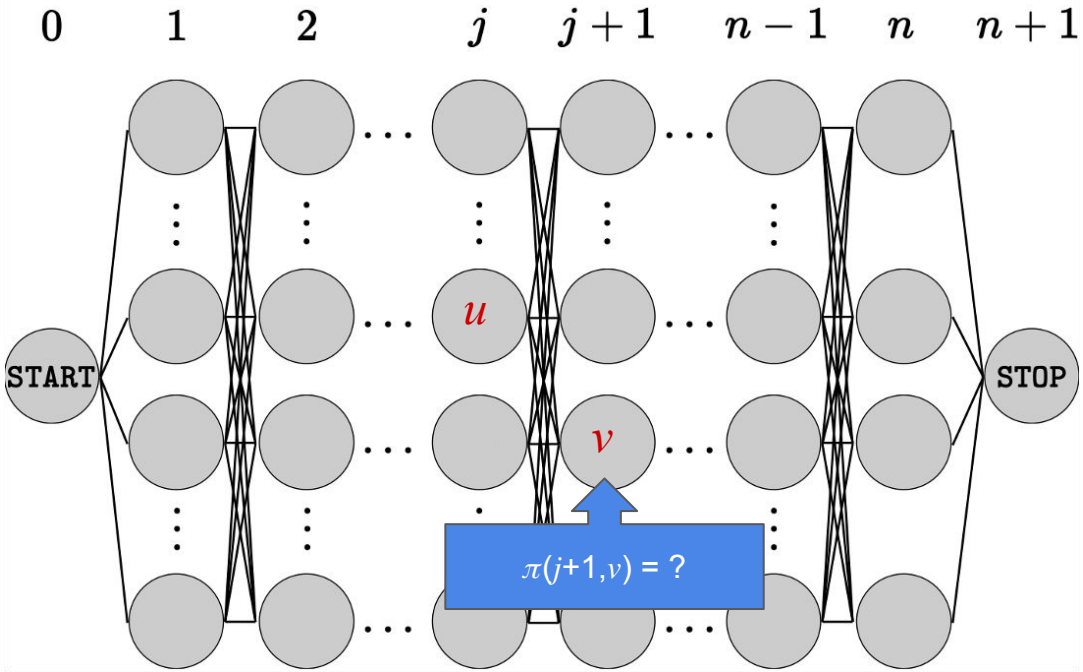
Initialized to 1
∴ it multiplies with all other probabilities.

$$\pi(0, \text{START}) = p(y_0 = \text{START})$$

Viterbi Algorithm

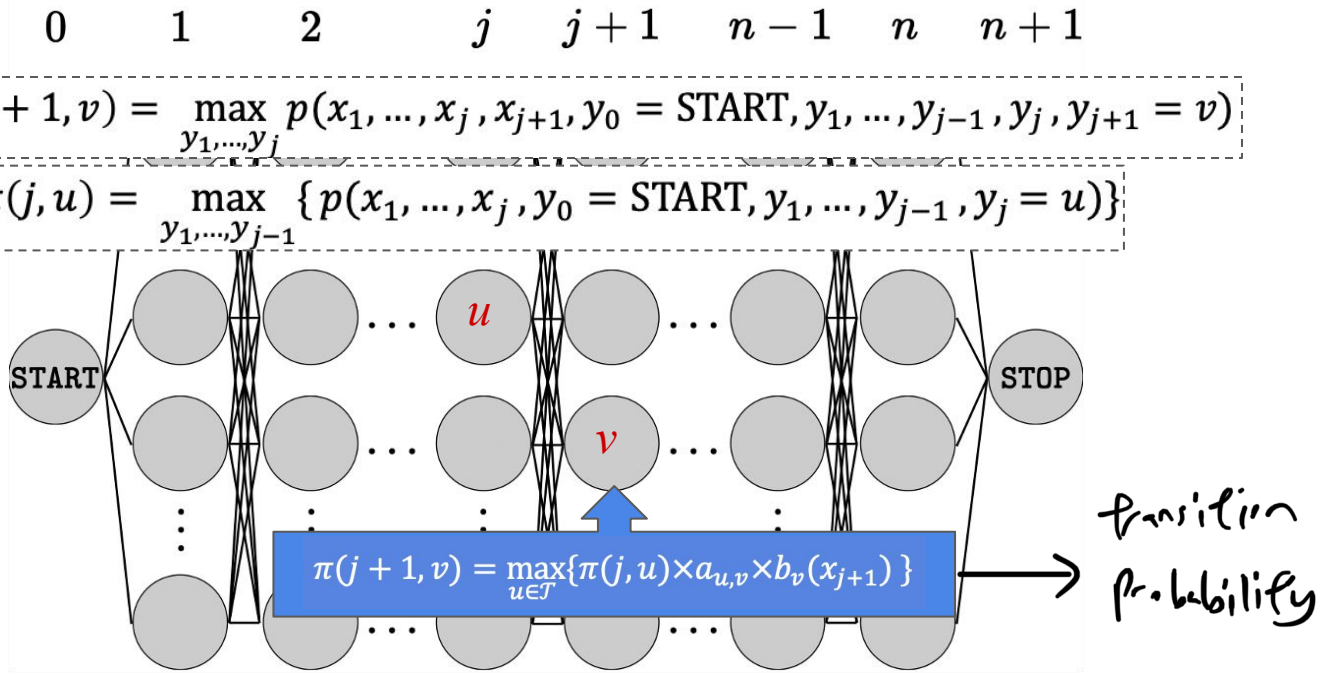


Viterbi Algorithm

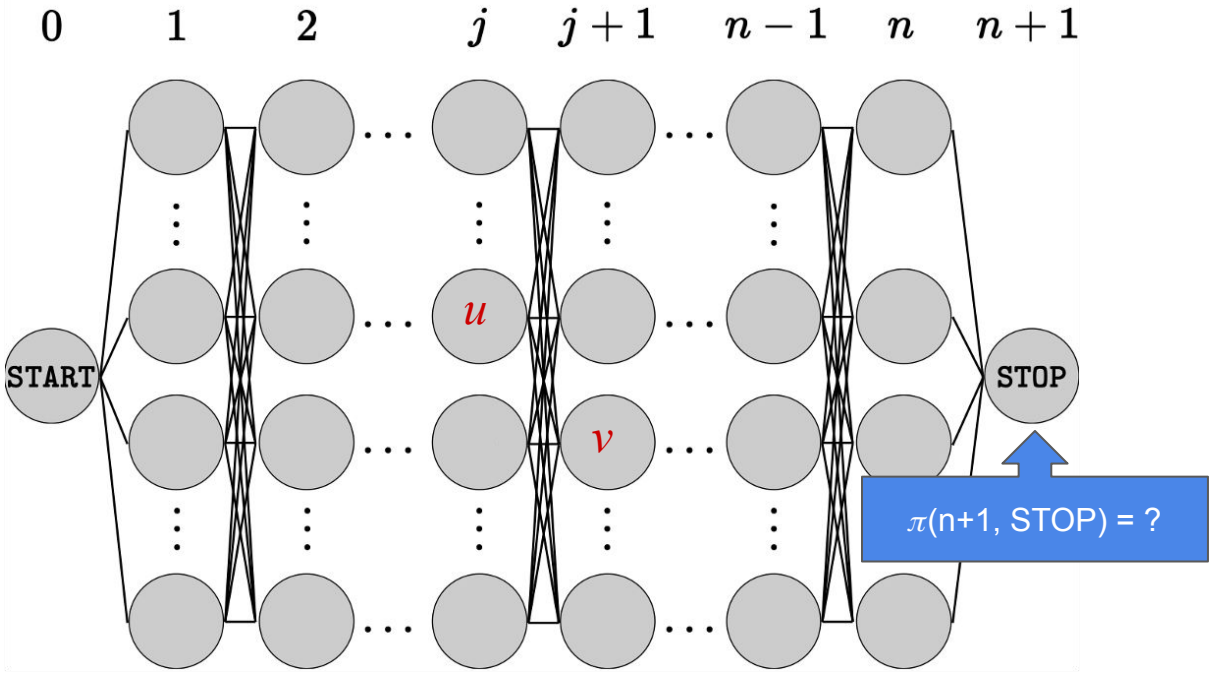


$$\pi(j+1, v) = \max_{y_1, \ldots, y_j} p(x_1, \ldots, x_j, x_{j+1}, y_0 = \text{START}, y_1, \ldots, y_{j-1}, y_j, y_{j+1} = v)$$

Viterbi Algorithm



Viterbi Algorithm



Viterbi Algorithm

*similar to Bellman-Ford

- Initialization

$$\pi(0,u) = \begin{cases} 1 & if\ u = \text{START} \\ 0 & otherwise \end{cases}$$

- For $j = 0, \ldots, n-1$:
  for each $v \in \mathcal{T}$:

$$\pi(j+1,v) = \max_{u \in \mathcal{T}} \{\pi(j,u) \times a_{u,v} \times b_v(x_{j+1})\}$$

- Final Step

$$\pi(n+1, \text{STOP}) = \max_{u \in \mathcal{T}} \{\pi(n,u) \times a_{u,\text{STOP}}\}$$

Viterbi Algorithm

- Initialization

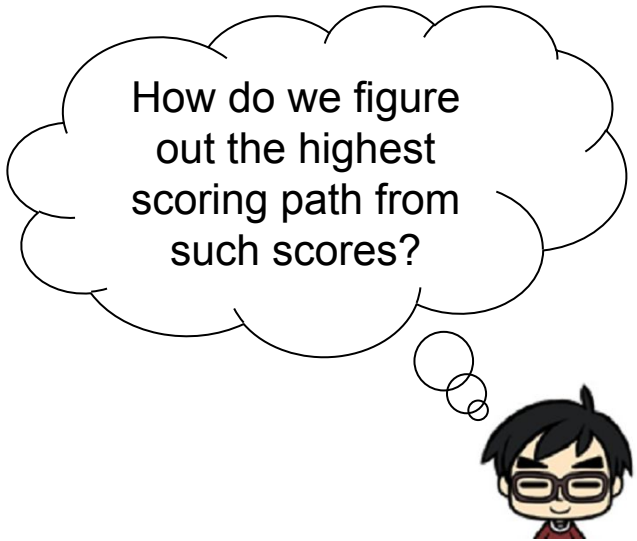
$$\pi(0,u) = \begin{cases} 1 & if\ u = \text{START} \\ 0 & otherwise \end{cases}$$

- For $j = 0, \ldots, n-1$:
  for each $v \in \mathcal{T}$:

$$\pi(j+1,v) = \max_{u \in \mathcal{T}} \{\pi(j,u) \times a_{u,v} \times b_v(x_{j+1})\}$$

- Final Step

$$\pi(n+1, \text{STOP}) = \max_{u \in \mathcal{T}} \{\pi(n,u) \times a_{u,\text{STOP}}\}$$



Viterbi Algorithm - Backtracking

Now, having values $\pi(j, u)$, we reconstruct the most likely sequence of tags $\mathbf{y}^* = \arg\max_{\mathbf{y}} p(\mathbf{x}, \mathbf{y})$ via **backtracking**:

- The best value for the last hidden random variable:

$$y_n^* = \underset{u \in \mathcal{T}}{\text{argmax}} \{\pi(n, u) \times a_{u,\text{STOP}}\}$$

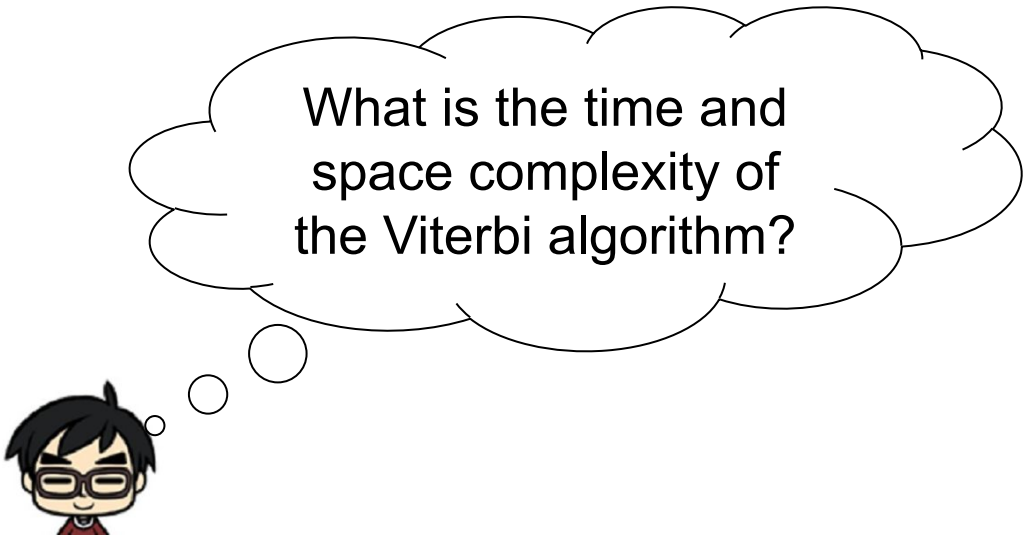
- For $j = n - 1, \ldots, 1$:

$$y_j^* = \underset{u \in \mathcal{T}}{\text{argmax}} \{\pi(j, u) \times a_{u,y_{j+1}^*} \times b_{y_{j+1}^*}(x_{j+1})\}$$

$$y_j^* = \underset{u \in \mathcal{T}}{\text{argmax}} \{\pi(j, u) \times a_{u,y_{j+1}^*}\}$$

Drop $b_{y_{j+1}^*}(x_{j+1})$ since it is irrelevant to u at j-th position.

Question


What is the time and space complexity of the Viterbi algorithm?

Viterbi Algorithm - Time & Space Complexity

- Initialization

$$\pi(0,u) = \begin{cases} 1 & if\ u = \text{START} \\ 0 & otherwise \end{cases}$$

- For $j = 0, \ldots, n-1$: n

  for each $v \in \mathcal{T}$: $|\mathcal{T}|$

  $$\pi(j+1,v) = \max_{u \in \mathcal{T}} \{\pi(j,u) \times a_{u,v} \times b_v(x_{j+1})\}$$ $|\mathcal{T}|$

  $n|\mathcal{T}|$

- Final Step

$$\pi(n+1, \text{STOP}) = \max_{u \in \mathcal{T}} \{\pi(n,u) \times a_{u,\text{STOP}}\}$$

Time Complexity: $O(n|\mathcal{T}|^2)$

Space Complexity: $O(n|\mathcal{T}|)$

What you should know?

- What is decoding for the HMM?
- Why do we need the Viterbi algorithm for decoding?
- How to perform decoding using the Viterbi algorithm?
- What is the space and time complexity of the Viterbi algorithm?

Suggestion: If you're not very clear or want to learn more, please do read: "C. Bishop: Pattern Recognition and Machine Learning. Springer, 2006" (recommended text book). It will help you to understand the concept.

Acknowledgement

- *The following material have been referenced or partially used:*
  - *Lecture notes from Prof. Lu Wei*
  - *Lecture notes from Prof. Zhao Na*
  - *National University of Singapore – Pattern Recognition Module (EE5907R)*
  - *National University of Singapore – Neural Networks Module (EE5904R)*
  - *University of Edinburgh, United Kingdom – Machine Learning And Pattern Recognition (MLPR, INFR11130)*
  - *Stanford University – Machine Learning (CS229)*